



厦门大学
XIAMEN UNIVERSITY

面向对象编程



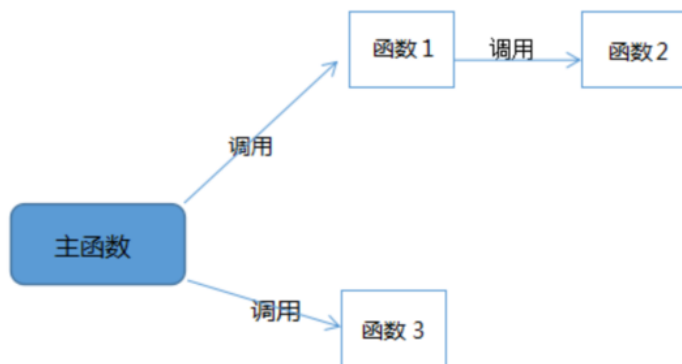
目录

1. 类和对象
2. 类的属性和方法
3. 访问限制
4. 类的继承

已知：农场有头大母牛，每年生头小母牛，小母牛五年后生小母牛，年龄大于15便死亡💀，请问：问20年后农场一共有多少头牛？

面向过程

- 分析出解决问题所需步骤
- 用函数把这些步骤一步一步实现
- 根据需要依次调用



面向过程

面向对象

- 把构成问题事务分解成各个对象
- 建立对象的目的是为了描述某个事物在整个解决问题的步骤中的行为



面向对象

1. 封装性

- 封装是指将一个计算机系统中的数据以及与这个数据相关的一切操作语言（即描述每一个对象的属性以及其行为的程序代码）组装到一起，封装在一个有机的实体中，也就是一个类中
- 为软件结构的相关部件所具有模块性提供良好的基础。
- 封装的最基本单位是对象
- 使得软件结构的相关部件的实现“高内聚、低耦合”的“最佳状态”便是面向对象技术的封装性所需要实现的最基本的目标。
- 对于用户来说，对象是如何对各种行为进行操作、运行、实现等细节是不需要了解清楚
- 用户只需要通过封装外的通道对计算机进行相关方面的操作即可

2、继承性：

- 继承性是面向对象技术中的另外一个重要特点，其主要指的是两种或者两种以上的类之间的联系与区别。
- 继承在面向对象技术则是指一个对象针对于另一个对象的某些独有的特点、能力进行复制或者延续。

按照继承源进行划分，则可以分为

- 单继承（一个对象仅仅从另外一个对象中继承其相应的特点）
- 与多继承（一个对象可同时从两个或两个以上的对象中继承特点与能力，且不会发生冲突

2、继承性：

如果从继承中包含的内容进行划分，则继承可以分为四类：

- 取代继承（一个对象在继承另一个对象的能力与特点之后将父对象进行取代）
- 包含继承（一个对象在将另一个对象的能力与特点进行完全的继承之后，又继承了其他对象所包含的相应内容，最终该对象所具有的能力与特点大于等于父对象，实现了对于父对象的包含）
- 受限继承
- 特化继承

3、多态性：

- 从宏观的角度来讲，多态性是指在面向对象技术中，当不同的多个对象同时接收到同一个完全相同的消息之后，所表现出来的动作是各不相同的，具有多种形态
- 从微观的角度来讲，多态性是指在一组对象的一个类中，面向对象技术可以使用相同的调用方式来对相同的函数名进行调用，即便这若干个具有相同函数名的函数所表示的函数是不同的

- 在面向对象编程中，我们编写表示现实世界中的事物和情景的类，并基于这些类来编写对象
- 对象是数据与相关行为的集合
- 类用来描述对象，以类为模板创建对象的过程称为类的实例化，这些对象又叫做实例

创建一个类



厦门大学
XIAMEN UNIVERSITY

格式:

class 类名称:

各种属性和方法

创建一个类的实例:

实例名称 = 类名(<参数表>)

```
class Dog:
    """Simulate a little
    dog"""
    def __init__(self, name, age):
        """Initiate name & age"""
        self.name = name
        self.age = age

    def sit(self):
        """Simulation when a dog is ordered to sit down"""
        print(self.name.title()+"is now sitting.")

    def roll_over(self):
        """Simulation when a dog is ordered to rollover"""
        print(self.name.title()+"rolled over")

my_dog = Dog('willie', 7)
my_dog.sit()
my_dog.roll_over()
```

- 属性：例如在前面的Dog类中，我们可以在类里面定义小狗的名字、行为等，这些叫做属性。
- 属性分为类属性和实例属性。
- 方法：例如在前面的Dog类中，我们可以在类里面定义打印小狗的名字、和动作等，这些叫做方法。
- 方法分为类方法和实例方法。类中的函数称为方法

定义实例的属性

定义方法：定义__init__函数，注意init左右两边各有两个下划线。

```
class Student:
    def __init__(self, name, number, score = 0):
        self.name = name
        self.number = number
        self.score = score
        self.age = 18
        self.city = "Xia Men"
```

在实例中，定义的每个函数都需要带有self参数，用于在函数体内部指明是该实例的属性（方法即为self. 属性名）

定义实例的属性

`__init__()` 一个特殊的方法:

- 下划线避免Python默认方法和普通方法发生命名冲突
- 包含self、name、age三个形参
- Python调用`__init__()`方法创建实例的时候，自动传入实参self
- 每个与类相关联的方法调用都自动传递实参self，它是一个指向实例本身的引用，让实例能够访问类中的属性和方法
- 以self为前缀的变量可供类中所有方法使用

```
class Dog:
```

定义一个名为Dog的类，首字母大写

```
"""Simulate a little dog"""
```

编写文档对类进行说明

```
def __init__(self,name,age):
```

```
"""Initiate name & age"""
```

```
self.name = name
```

```
self.age = age
```

Dog类创建了两个方法

```
sit()
```

```
roll_over()
```

```
def sit(self):
```

```
"""Simulation when a dog is ordered to sit down"""
```

```
print(self.name.title()+"is now sitting.")
```

```
def roll_over(self):
```

```
"""Simulation when a dog is ordered to rollover"""
```

```
print(self.name.title()+"rolled over")
```

- `self.name = name` 获取存储在形参name中的值，并将其存储到变量name中，然后该变量被关联到当前创建的实例
- `self.age=age`的作用与name 同
- 像这样可通过实例访问的变量称为属性。

```
my_dog = Dog('willie', 7)
print("My dog's name
is"+my_dog.name.title()+".")
print("My dog
is"+str(my_dog.age)+"years old.")
my_dog.sit()
my_dog.roll_over()
```

根据类创建实例

访问属性 “.”

```
your_dog = Dog('lucy', 3)
print("\nYour dog's name
is"+your_dog.name.title()+".")
print("Your dog
is"+str(your_dog.age)+"years old.")
your_dog.sit()
```

创建多个实例

属性和方法



厦门大学
XIAMEN UNIVERSITY

```
class CarOdometer:
```

```
"""一次模拟汽车的简单尝试"""
```

```
def __init__(self, make, model, year):
```

```
    """初始化汽车描述"""
```

```
    self.make = make
```

```
    self.model=model
```

```
    self.year=year
```

```
    self.odometer_reading = 0
```

```
my_new_car = CarOdometer('audi', 'a4', 2016)
```

```
print(my_new_car.get_describe_name())
```

```
my_new_car.read_odometer()
```

给属性指定默认值

```
def get_describe_name(self):
```

```
    """返回整体的描述性信息"""
```

```
    long_name=str(self.year)+' '+self.make+' '+self.model
```

```
    return long_name.title()
```

```
def read_odometer(self):
```

```
    """打印出一条汽车里程的消息"""
```

```
    print("This car
```

```
has" +str(self.odometer_reading)+ "miles on it.")
```

添加方法读取odometer

三种修改属性的方法

"""修改属性的值：1直接修改"""

```
print("Change attribute directly:")  
my_new_car.odometer_reading = 23  
my_new_car.read_odometer()
```

```
Change attribute directly:  
This car has 23 miles on it.
```

三种修改属性的方法

"""修改属性的值：2通过方法修改属性的值"""

```
class CarOdometer1:
```

```
"""一次模拟汽车的简单尝试"""
```

```
    def __init__(self, make, model, year):
```

```
        """初始化汽车描述"""
```

```
...
```

```
    def update_odometer(self, mileage):
```

```
        """将里程表的读数设定为指定的值"""
```

```
        self.odometer_reading=mileage
```

```
my_new_car = CarOdometer1('audi', 'a4', 2016)
```

```
print("Change attribute by method:")
```

```
my_new_car.update_odometer(25)
```

```
my_new_car.read_odometer()
```

添加update_odometer()方法

该方法接受一个里程值，并将其存储到
self.odometer_reading中

调用update_odometer(),
并向其提供实参23

三种修改属性的方法

```
# 3 通过方法对属性的值进行递增
"""increase attribute by method"""

class CarOdometer2:
    """一次模拟汽车的简单尝试"""
    ...
    def update_odometer(self, mileage):
        """将里程表的读数设定为指定的值"""
        if mileage >= self.odometer_reading:
            self.odometer_reading = mileage

    def increment_odometer(self, miles):
        """将里程表数据增加"""
        self.odometer_reading += miles

my_used_car.increment_odometer(1000)
my_used_car.read_odometer()
```

定义类的属性

类属性（相当于C++的静态数据成员）的定义：在__init__函数之外定义，这个属性是所有类的实例共享的，比如在Student类内定义一个类属性average，存储所有学生的平均分，每个实例共享之。

```
class Student:

    count = 0 # 记录人数
    average = 0.0 # 记录平均分

    def __init__(self, name, score = 0):
        self.name = name
        self.score = score
        Student.count += 1
        Student.average = (Student.average + score) / Student.count

student_a = Student('Alice', 120)
student_b = Student('Bob', 100)
print(student_a.average)
```

110.0

进程已结束, 退出代码为 0

类属性可以直接通过
类名. 属性名访问，也可以
通过实例. 属性名访问。

定义实例的方法

定义方法：即定义函数的方法，注意第一个参数必须是self

```
def get_score(self):  
    print("{}'s score is {}".format(self.name, self.score))  
def reset_name(self, new_name):  
    self.name = new_name
```

在实例中使用方法：实例名.方法名(<参数表>)

```
student1 = Student('Alice', 123)  
student1.get_score()  
student1.reset_name('Bob')  
student1.get_score()
```

Alice's score is 0

Bob's score is 0

进程已结束，退出代码为 0

定义类的方法(拓展)

类方法：在用@classmethod绑定方法，使其称为类方法，函数的第一个参数必须是cls，用于指向类本身，比如定义一个打印学生平均分的类方法：

```
@classmethod  
def get_average(cls):  
    print(cls.average)
```

```
student_a = Student('Alice', 120)  
student_b = Student('Bob', 100)  
Student.get_average()
```

110.0

进程已结束，退出代码为 0

同样，实例也可以使用类方法。

- 编写类时，可不用从空白开始。如要编写的类是另一个现成类的特殊版本，可使用继承。
- 一个类继承另一个类时，将自动获得另一个类的所有属性和方法
- 原有的类称为父类，而新类称为子类
- 子类继承了其父类的所有属性和方法，同时还可以定义自己的属性和方法

定义子类时，必须在括号内指定父类的名称。
方法 `__init__()` 接受创建Car实例所需的信息

```
class Car3:
    """一次模拟汽车的简单尝试"""
    ...
    """下面实现继承"""
```

首先是Car类的代码
创建子类时，父类必须包含在当前文件中，且位于子类前面

`super()` 是一个特殊函数，帮助Python将父类和子类关联起来

```
class ElectricCar(Car3):
    """电动汽车的独特之处"""
    def __init__(self, make, model, year):
        """初始化父类的属性"""
        super().__init__(make, model, year)
```

- 代码让Python调用ElectricCar的父类的方法 `__init__()`，
- 让ElectricCar实例包含父类的所有属性
- 父类也称为超类（superclass），名称super因此而得名。

```
my_tesla=ElectricCar('tesla','models',2016)
print(my_tesla.get_describe_name())
```

""" 给子类定义属性和方法 """

```
class Car:
```

```
...
```

```
class ElectricCar(Car):
```

```
    """电动汽车的独特之处"""
```

```
    def __init__(self, make, model, year):
```

```
        """初始化父类的属性"""
```

```
        super().__init__(make, model, year)
```

```
        self.battery_size = 70
```

```
    def describe_battery(self):
```

```
        """打印一条描述电瓶容量的消息"""
```

```
        print("This car has a " + str(self.battery_size) + " -
```

```
kwh battery")
```

```
my_tesla = ElectricCar('tesla', 'model s', 2016)
```

```
print(my_tesla.get_describe_name())
```

```
my_tesla.describe_battery()
```

- 添加了新属性 self.battery_size, 并设置其初始值 (如 70)
- 根据 ElectricCar 类创建的所有实例都将包含这个属性
- 所有 Car 实例都不包含它
- 添加了一个名为 describe_battery() 的方法, 它打印有关电瓶的信息
- 我们调用这个方法时, 将看到一条电动汽车特有的信息

重写父类的方法

- 对于父类的方法，只要它不符合子类模拟的实物的行为，都可对其进行重写
- 可在子类中定义一个这样的方法，即它与要重写的父类方法同名
- Python 将不会考虑这个父类方法，而只关注在子类中定义的相应方法

假设Car 类有一个名为fill_gas_tank() 的方法，它对全电动汽车来说毫无意义，因此你可能想重写它

重写父类的方法

```
class Car:
    """一次模拟汽车的简单尝试"""
    def fill_gas_tank(self):
        """为汽车加油"""
        print(" This car needs to
gas up !")
```

```
class ElectricCar(Car):
    """电动汽车的独特之处"""
    ...
# 以下在子类中重写父类的方法:
    def fill_gas_tank(self):
        """电动汽车没有油箱"""
        print(" This car doesn't need a
gas tank !")

my_tesla = ElectricCar('tesla', 'model
s', 2016)
print(my_tesla.get_describe_name())
my_tesla.describe_battery()
my_tesla.fill_gas_tank()
```



```
class Car:
    ...
class Battery:
    """一次模拟电动汽车电瓶的简单尝试"""

    def __init__(self, battery_size = 70):
        """初始化电瓶的属性"""
        self.battery_size = battery_size

    def describe_battery(self):
        """打印一条描述电瓶容量的消息f"""
        print("This car has a " +
              str(self.battery_size)+ "-kwh battery.")
```

- 定义了一个名为Battery的新类， 它没有继承任何类
- 方法__init__() 除self外，还有另一个形参battery_size
- 这形参是可选的，如没给它提供值，电瓶容量将被设置为70
- 方法 describe_battery() 也移到了这个类中

```
class ElectricCar(Car):
    """电动汽车的独特之处"""
    def __init__(self, make, model, year):
        """初始化父类的属性"""
        super().__init__(make, model, year)
        self.battery = Battery()

    def describe_battery(self):
        """打印一条描述电瓶容量的消息"""
        print("This car has a " + str(self.battery_size) + "-kwh
battery")

my_tesla = ElectricCar('tesla', 'model s', 2016)
print(my_tesla.get_describe_name())
my_tesla.battery.describe_battery()
```

- 在 `ElectricCar` 类中，我们添加了一个名为 `self.battery` 的属性
- 这行代码让Python创建 一个新的 `Battery` 实例（ 由于没有指定尺寸，因此为默认值70）
- 并将 该 实例存储在属性 `self.battery` 中，每当方法 `__init__()` 被调用时，都将执行该操作
- 因此，现在每个 `ElectricCar` 实例都包含一个自动创建的 `Battery` 实例

类的私有属性

`__private_attrs:`

- 两个下划线开头，声明该属性为私有
 - 不能在类的外部被使用或直接访问
- 在类内部的方法中使用时

`self.__private_attrs`

类的方法

- 在类的内部，使用 `def` 关键字来定义一个方法，与一般函数定义不同，类方法必须包含参数 `self`，且为第一个参数，`self` 代表的是类的实例
- `self` 的名字并不是规定死的，也可以使用 `this`，但是最好还是按照约定使用 `self`。

```
Class JustCounter:
```

```
    __secretCount = 0 # 私有变量  
    publicCount = 0  # 公开变量
```

```
    def count(self):  
        self.__secretCount += 1  
        self.publicCount += 1  
        print (self.__secretCount)
```

```
counter = JustCounter()  
counter.count()  
counter.count()  
print (counter.publicCount)  
print (counter.__secretCount) # 报错，实例不能访问私有变量
```

类的私有方法

`__private_method:`

- 两个下划线开头，声明该方法为私有方法，只能在类的内部调用，不能在类的外部调用
- `self.__private_methods`

```
1  
2  
2
```

Traceback (most recent call last):

File "test.py", line 16, in <module>

print (counter.__secretCount) # 报错，实例不能访问私有变量

AttributeError: 'JustCounter' object has no attribute '__secretCount'

```
class Site:
    def __init__(self, name, url):
        self.name = name        # public
        self.__url = url       # private

    def who(self):
        print('name : ', self.name)
        print('url : ', self.__url)

    def __foo(self):             # 私有方法
        print('这是私有方法')

    def foo(self):              # 公共方法
        print('这是公共方法')
        self.__foo()
```

```
x = Site('auto-drive', '10.24.94.46')
x.who()           # 正常输出
x.foo()           # 正常输出
x.__foo()         # 报错
```

类的专有方法

`__init__` : 构造函数, 在生成对象时调用

`__del__` : 析构函数, 释放对象时使用

`__repr__` : 打印, 转换

`__setitem__` : 按照索引赋值

`__getitem__` : 按照索引获取值

`__len__` : 获得长度

`__cmp__` : 比较运算

`__call__` : 函数调用

`__add__` : 加运算

`__sub__` : 减运算

`__mul__` : 乘运算

`__truediv__` : 除运算

`__mod__` : 求余运算

`__pow__` : 乘方

类的继承的总结和拓展

在定义一个新类时，可以直接从原有的类中复制（继承）其属性和方法。

其中原来的类叫父类或基类，新定义的类叫做子类或派生类。

继承的方法：class派生类名(基类名1, 基类名2, ...):

从多个基类派生的，称为多继承

注意基类的顺序，在派生类继承基类的属性时，如果基类1和基类2都有共同的属性或方法名，派生类优先继承前者的。

类的继承的总结和拓展

给派生类添加额外属性时，要声明对基类__init__的调用。

方法：super().__init__(<参数表>)

```
class Father:
    def __init__(self, data3):
        self.data1 = 10
        self.data2 = 100
        self.data3 = data3

class Son(Father):
    def __init__(self, data3, data5):
        super().__init__(data3)
        self.data4 = 1000
        self.data5 = data5
```

参数表中要带上传入基类__init__()的参数

我们之前学过，对于+号运算符，如果是两个int类型相加，就是进行普通的整数相加，如果是两个str类型相加，就是将两个字符串连接起来，实际上str类型中对+号运算符进行了重载，改变了其功能。

Python的运算符重载与C++不同，重载运算符需要定义运算符对应的函数方法名，下面列举一些常用运算符的方法名：

方法名	表达式	说明
<code>__add__(self, hs)</code>	<code>self+hs</code>	加号
<code>__sub__(self, hs)</code>	<code>self-hs</code>	减号
<code>__mul__(self, hs)</code>	<code>self*hs</code>	乘号
<code>__truediv__(self, hs)</code>	<code>self/hs</code>	除号
<code>__floordiv__(self, hs)</code>	<code>self//hs</code>	整除
<code>__mod__(self, hs)</code>	<code>self%hs</code>	取模
<code>__pow__(self, hs)</code>	<code>self**hs</code>	乘幂

例子：定义一个复数类，重载+与*运算。

```
class Complex:
    def __init__(self, real = 0, image = 0):
        self.real = real
        self.image = image

    def __add__(self, other):
        temp_complex = Complex()
        temp_complex.real = self.real + other.real
        temp_complex.image = self.image + other.image
        return temp_complex

    def __mul__(self, other):
        temp_complex = Complex()
        temp_complex.real = self.real * other.real - self.image * other.image
        temp_complex.image = self.real * other.image + self.image * other.real
        return temp_complex

    def print(self):
        if self.image > 0:
            print("{}+{}i".format(self.real, self.image))
        elif self.image < 0:
            print("{}-{}i".format(self.real, abs(self.image)))
        else:
            print(self.real)
```

```
complex1 = Complex(2, 3)
complex2 = Complex(5, -9)
complex3 = complex1 + complex2
complex4 = complex1 * complex2
complex3.print()
complex4.print()
```

7-6i
37-3i

进程已结束，退出代码为 0



```
class Vector:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def __str__(self):
        return 'Vector (%d, %d)' % (self.a,
self.b)

    def __add__(self, other):
        return Vector(self.a + other.a,
self.b + other.b)

v1 = Vector(2, 10)
v2 = Vector(5, -2)
print (v1 + v2)

Vector(7, 8)      ? ? ?
```

在类的外部，我们可以轻易修改一个类或实例的属性：

```
class DemoClass:
    def __init__(self):
        self.data = 100
```

100
50

```
an_object = DemoClass()
print(an_object.data)
an_object.data = 50
print(an_object.data)
```

进程已结束，退出代码为 0

在实际应用中，我们并不希望外部能直接更改类或实例的属性，以避免运行产生错误。此时可以在属性前加入下划线，这样外部就不能访问了。

```
def __init__(self, name, score = 0):  
    self.__name = name  
    self.__score = score
```

Bob
Alice

```
def print(self):  
    print(self.__name)
```

进程已结束，退出代码为 0

```
student_a = Student('Alice', 120)  
student_a.name = 'Bob' # 试图直接修改名字  
print(student_a.name)  
student_a.print()
```

为什么是这个结果？

```
def __init__(self, name, score = 0):
```

```
    self.__name = name
```

```
    self.__score = score
```

在属性名前加两个下划线，它就变成了私有的，外部不能直接访问

```
def print(self):
```

```
    print(self.__name)
```

```
student_a = Student('Alice', 120)
```

```
student_a.name = 'Bob' # 试图直接修改名字
```

```
print(student_a.name)
```

```
student_a.print()
```

这一步实际上是新定义了一个名为student_a.name的变量，所以输出的结果是Bob

在Python中，访问了私有的实例属性不会报错，而是将其当作一个新定义的变量名来看待。

类或实例的属性实际上有三个类型：

分别为公有的(public)，受保护的(protected)，私有的(private)。

- 定义受保护属性只需在属性名前加入一个下划线即可，除了类与其派生类可以访问，其他部分不可访问；
- 定义私有属性只需在属性名前加入两个下划线即可，除了类自身可以访问，其他部分不可以访问。
- 其余属性均为公有属性（包括名称前后都有下划线的属性）

例子：定义一个员工类，其属性有姓名，年龄，性别。由其派生出一个经理类，额外添加职位属性。二者都有一个同样的方法名print。

```
class Employees:
    def __init__(self, name, age, sex):
        self.name = name
        self.age = age
        self.sex = sex
    def print(self):
        print("{}'s age is {}".format(self.name, self.age))
        print("{}'s sex is {}".format(self.name, self.sex))

class Manager(Employees):
    def __init__(self, name, age, sex, position):
        super().__init__(name, age, sex)
        self.position = position
    def print(self):
        print("{} is manager.".format(self.name))
        print("{}'s age is {}".format(self.name, self.age))
        print("{}'s sex is {}".format(self.name, self.sex))
        print("{}'s position is {}".format(self.name, self.position))
```

```
an_employee = Employees('Alice', 20, "female")
a_manager = Manager('Bob', 30, 'male', 'marketing')
an_employee.print()
a_manager.print()
```

```
Alice's age is 20
Alice's sex is female
Bob is manager.
Bob's age is 30
Bob's sex is male
Bob's position is marketing
```

面向对象技术小结:



厦门大学
XIAMEN UNIVERSITY

类(Class):用来描述具有相同的属性和方法的对象的集合。它定义了该集合中每个对象所共有的属性和方法。对象是类的实例。

类变量: 类变量在整个实例化的对象中是公用的。类变量定义在类中且在函数体之外。类变量通常不作为实例变量使用。

数据成员: 类变量或者实例变量,用于处理类及其实例对象的相关的数据。

方法重写: 如果从父类继承的方法不能满足子类的需求,可以对其进行改写,这个过程叫方法的覆盖(override),也称为方法的重写。

局部变量: 定义在方法中的变量,只作用于当前实例的类。

实例变量: 在类的声明中,属性是用变量来表示的。这种变量就称为实例变量,是在类声明的内部但是在类的其他成员方法之外声明的。

继承: 即一个派生类(derivedclass)继承基类(baseclass)的字段和方法。继承也允许把一个派生类的对象作为一个基类对象对待。例如,有这样一个设计:一个Dog类型的对象派生自Animal类,这是模拟"是一个(is-a)"关系(例图, Dog是一个Animal)。

实例化: 创建一个类的实例,类的具体对象。

方法: 类中定义的函数。

对象: 通过类定义的数据结构实例。对象包括两个数据成员(类变量和实例变量)和方法。

- 文件是一个存储在辅助存储器上的数据序列， 可以包含任何数据内容。
- 概念上， 文件是数据的集合和抽象， 函数是程序的集合和抽象。
- 用文件形式组织和表达数据更有效也更为灵活。
- 文件包括两种类型：文本文件和二进制文件
- 二进制文件直接由比特0和比特1组成， 没有统一字符编码，
- 文件内部数据的组织格式与文件用途有关
- 二进制文件和文本文件最主要的区别在于是否有统一的字符编码
- 无论文件创建为文本文件或者二进制文件， 都可以用“文本文件方式” 和“二进制文件方式” 打开， 打开后的操作不同
- 采用文本方式读入文件， 文件经过编码形成字符串， 打印出有含义的字符
- 采用二进制方式打开文件， 文件被解析为字节（ byte） 流。由于存在编码， 字符串中的一个字符由2个字节表示

```
textFile = open("file_open.txt", "rt", encoding='utf-8') # t 表示文本文件方式
print(textFile.readline())
textFile.close()
binFile = open("file_open.txt", "rb") # r 表示二进制文件方式
print(binFile.readline())
binFile.close()
```

人生这条路很长，未来如星辰大海般璀璨，不必踟躇与过去的半亩方塘。那些所谓的遗憾，可能是一种成长那些曾受过的伤，终会化作照亮前路的光！

```
b' \xe4\xba\xba\xe7\x94\x9f\xe8\xbf\x99\xe6\x9d\xa1\xe8\x
b7\xaf\xe5\xbe\x88\xe9\x95\xbf\xef\xbc\x8c\xe6\x9c\...
```

文件的打开关闭

Python对文本文件和二进制文件采用统一的操作步骤，即“打开-操作-关闭”

Python通过解释器内置的`open()`函数打开一个文件，并实现该文件与一个程序变量的关联

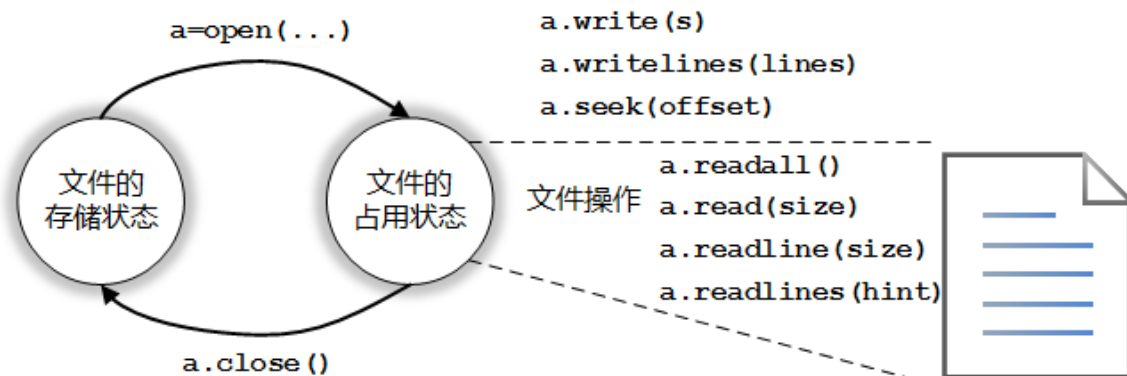
`open()`函数格式如下：

`<变量名> = open(<件名>, <打开模式>)`

`open()`函数有两个参数：**文件名**和**打开模式**

文件名可以是文件的实际名字，也可以是包含完整路径的名字

`open()`函数提供7种基本的打开模式



打开模式	含义
'r'	只读模式，如果文件不存在，返回异常 <code>FileNotFoundError</code> ，默认值
'w'	覆盖写模式，文件不存在则创建，存在则完全覆盖源文件
'x'	创建写模式，文件不存在则创建，存在则返回异常 <code>FileExistsError</code>
'a'	追加写模式，文件不存在则创建，存在则在原文件最后追加内容
'b'	二进制文件模式
't'	文本文件模式，默认值
'+'	与r/w/x/a一同使用，在原功能基础上增加同时读写功能

根据打开方式不同可以对文件进行相应的读写操作，Python提供4个常用的文件内容读取方法：

方法	含义
<code><file>.readall()</code>	读入整个文件内容，返回一个字符串或字节流*
<code><file>.read(size=-1)</code>	从文件中读入整个文件内容，如果给出参数，读入前size长度的字符串或字节流
<code><file>.readline(size = -1)</code>	从文件中读入一行内容，如果给出参数，读入该行前size长度的字符串或字节流
<code><file>.readlines(hint=-1)</code>	从文件中读入所有行，以每行为元素形成一个列表，如果给出参数，读入hint行

```
file_name = input("请输入要打开的文件：")
fo = open(file_name, "r")
for line in fo.readlines():
    print(line)
fo.close()
```

如果程序需要逐行处理文件内容，建议采用如下代码格式：

```
fo = open(fname, "r")
for line in fo:
    # 处理一行数据
fo.close()
```

Python提供3个与文件内容写入有关的方法

方法	含义
<code><file>.write(s)</code>	向文件写入一个字符串或字节流
<code><file>.writelines(lines)</code>	将一个元素为字符串的列表写入文件
<code><file>.seek(offset)</code>	改变当前文件操作指针的位置， <code>offset</code> 的值： 0：文件开头； 1: 当前位置； 2: 文件结尾

```
fname = input("请输入要写入的文件：")
fo = open(fname, "w+")
ls = ["唐诗", "宋词", "元曲"]
fo.writelines(ls)
for line in fo:
    print(line)
fo.close()
```


数据组织的维度

- 一维数据由对等关系的有序或无序数据构成，采用线性方式组织，对应于数学中的数组和集合等概念

中国、 美国、 日本、 德国、 法国、 英国、 意大利、 加拿大、 俄罗斯、 欧盟、 澳大利亚、 南非、 阿根廷、 巴西、 印度、 印度尼西亚、 墨西哥、 沙特阿拉伯、 土耳其、 韩国

- 一维数据是最简单的数据组织类型， 有多种存储格式， 常用特殊字符分隔：

（ 1） 用一个或多个空格分隔， 例如： 中国 美国 日本 德国 法国 英国 意大利

（ 2） 用逗号分隔， 例如： 中国, 美国, 日本, 德国, 法国, 英国, 意大利

（ 3） 用其他符号或符号组合分隔， 建议采用不出现在数据中的特殊符号
中国； 美国； 日本； 德国； 法国； 英国； 意大利

二维数据 CSV格式

逗号分割数值的存储格式叫做CSV格式（CommaSeparated Values，即逗号分隔值），它是一种通用的、相对简单的文件格式，在商业和科学上广泛应用，尤其应用在程序之间转移表格数据

该格式的应用有一些基本规则，如下：

- 纯文本格式，通过单一编码表示字符；
- 以行为单位，开头不留空行，行之间没有空行；
- 每行表示一个一维数据，多行表示二维数据；
- 以逗号分隔每列数据，列数据为空也要保留逗号；
- 可以包含或不包含列名，包含时列名放置在文件第一行。

二维数据

- CSV格式存储的文件一般采用.csv为扩展名,
- 可通过记事本或Excel工具打开
- 也可在其他操作系统平台上用文本编辑工具打开

- CSV文件的每一行是一维数据, 可以使用Python中的列表类型表示
- 整个CSV文件是一个二维数据, 由表示每一行的列表类型作为元素, 组成一个二维列表

二维数据采用CSV存储后的内容如下:

城市	环比	同比	定基
北京	101.5	120.7	121.4
上海	101.2	127.3	127.8
广州	101.3	119.4	120
深圳	102	140.9	145.5
沈阳	100.1	101.4	101.6

```
fo = open("price2016.csv",  
"r", encoding='UTF-8')  
for line in fo:  
    ls = []  
    line = line.replace("\n",  
    "")  
    ls.append(line.split(","))  
    print(ls)  
fo.close()
```

```
[['城市', '环比', '同比', '定基']]  
[['北京', '101.5', '120.7', '121.4']]  
[['上海', '101.2', '127.3', '127.8']]  
[['广州', '101.3', '119.4', '120']]  
[['深圳', '102', '140.9', '145.5']]  
[['沈阳', '100.1', '101.4', '101.6']]
```

对于列表中存储的二维数据， 可以通过循环写入一维数据的方式写入CSV文件， 参考代码样式如下：

```
for row in ls:  
<输出文件>.write(",".join(row)+"\n")
```

高维数据的格式化

- 与一维二维数据不同，高维数据能展示数据间更为复杂的组织关系。
- 为了保持灵活性，表示高维数据不采用任何结构形式，仅采用最基本的二元关系，即键值对
- 万维网是高维数据最成功的典型应用
- JSON格式可以对高维数据进行表达和存储。
- JSON (JavaScript Object Notation) 是一种轻量级的数据交换格式，易于阅读和理解。
- JSON格式 表达键值对<key, value>的基本格式如下，键值对都保存在双引号中：
- "key" : "value"

高维数据的格式化

当多个键值对放在一起时，JSON有如下约定：

- 数据保存在键值对中；
- 键值对之间由逗号分隔；
- 大括号用于保存键值对数据组成的对象；
- 方括号用于保存键值对数据组成的数组。

```
“本门课程”： [  
{ “课程名称”： “智能驾驶”，  
  “学期”： “2022春季”，  
  “单位”： “厦门大学” },  
{ “开课人”： “谢作生”，  
  “开课学院”： “信息学院”，  
  “单位”： “厦门大学” },  
]
```

json库

json库主要包括两类函数：操作类函数和解析类函数

- 操作类函数主要完成外部JSON格式和程序内部数据类型之间的转换功能
- 解析类函数主要用于解析键值对内容

`dumps()` 和 `loads()` 分别对应编码和解码功能

函数	描述
<code>json.dumps(obj, sort_keys=False, indent=None)</code>	将Python的数据类型转换为JSON格式，编码过程
<code>json.loads(string)</code>	将JSON格式字符串转换为Python的数据类型，解码过程
<code>json.dump(obj, fp, sort_keys=False, indent=None)</code>	与 <code>dumps()</code> 功能一致，输出到文件 <code>fp</code>
<code>json.load(fp)</code>	与 <code>loads()</code> 功能一致，从文件 <code>fp</code> 读入

将CSV转换成JSON格式

```
import json
fr = open("price2016.csv", "r", encoding='UTF-8')
ls = [ ]
for line in fr:
    line = line.replace("\n", "")
    ls.append(line.split(','))
fr.close()
fw = open("price2016.json", "w")
for i in range(1, len(ls)):
    ls[i] = dict(zip(ls[0], ls[i]))
json.dump(ls[1:], fw, sort_keys=True, indent=4,
ensure_ascii=False)
fw.close()
```

JSON格式数据转换成CSV格式

```
import json
fr = open("price2016.json", "r")
ls = json.load(fr)
data = [ list(ls[0].keys()) ]
for item in ls:
    data.append(list(item.values()))
fr.close()
fw = open("price2016_from_json.csv", "w")
for item in data:
    fw.write(",".join(item) + "\n")
fw.close()
```


万维网（WWW）的快速发展带来了大量获取和提交网络信息的需求，这产生了“网络爬虫”等一系列应用。

Python 语言提供了很多类似的函数库，包括urllib、urllib2、urllib3、wget、scrapy、requests 等。这些库作用不同、使用方式不同、用户体验不同

对于爬取回来的网页内容，可以通过re（正则表达式）、beautifulsoup4等函数库来处理，最重要且最主流的两个函数库：requests 和beautifulsoup4，它们都是第三方库

网络爬虫应用一般分为两个步骤：

- （1）通过网络连接获取网页内容
- （2）对获得的网页内容进行处理。

这两个步骤分别使用不同的函数库：requests 和 beautifulsoup4

```
conda install requests
```

```
conda install beautifulsoup4
```

获得一个HTML页面的通用代码

```
import requests
def getHTMLText():
    try:
        r = requests.get(url, timeout=30)
        r.raise_for_status() #如果状态不是200, 引发异常
        r.encoding = 'utf-8' #无论原来用什么编码, 都改成utf-8
        return r.text
    except:
        return ""
url = "http://www.baidu.com"
print(getHTMLText(url))
```